

Build spec — underwriting rule-to-code POC pipeline

Compiled from a full Gru pass, /v0 through /p5. Every decision below was made explicitly, in sequence, with the reasoning preserved. Nothing here should be silently renegotiated during implementation — if a decision needs to change, reopen the phase it belongs to, don't just overwrite it in code.

v0 — Mission

A proof-of-concept multi-agent pipeline that translates plain-English underwriting/eligibility rules into Python code, where separate agents generate, validate (against the stated rule, not legal/compliance standards), and simulate that code on synthetic applicant data, culminating in a human reviewing the linked code-and-simulation-result pair for translation fidelity — explicitly not evaluated for fair-lending or production readiness.

Phase 0 — Vision

Users (by role, not headcount — all currently the same person):

- Rule-author: writes the plain-English rule
- Oracle-author: hand-writes expected outcomes per synthetic applicant, *before* code generation, independent of any agent's interpretation
- Reviewer/approver: reviews the linked code + simulation + oracle comparison
- Future stranger (provisioned for, not currently present): must be able to audit disagreements using only rationale fields, no shared context

Business/portfolio need:

Demonstrate, through a working multi-agent pipeline, that naive trust in AI-generated code is unsafe in high-stakes domains — and that structural separation (independent human oracle predating code generation, validation scoped to translation-fidelity rather than correctness-of-judgment, and human approval of a linked code+simulation pair) is a viable defense.

Thesis (B) is the point. Orchestration (A) is the evidence for the thesis, not the thesis itself. When these conflict, rigor wins — no exceptions, no "just this once."

Success criteria:

- End-to-end run completes (per failure policy in Phase 1)
- Every agent output includes rationale, not just verdict — legible to a stranger

- At least one rule contains a realistic (non-contrived) ambiguity that generated code plausibly mishandles, caught by the oracle/simulation comparison
- Oracle written and frozen before code generation, every rule, no exceptions
- Approved code and approved simulation result are structurally linked (tamper-evident)

Constraints / non-goals (locked):

- No real applicant data, ever
 - No fair-lending / disparate-impact / compliance evaluation — validation is translation-fidelity only
 - No production deployment path, no "go-live"
 - No agent computes or influences oracle content
 - No merging generate/validate roles, ever — this is a thesis violation, not a simplification
 - Single Python process, agents as functions/classes — logical separation, not infrastructural
 - Attended operation — you run and narrate; no unattended execution, no defensive UX for a solo stranger-operator
 - "Multi-agent" must be qualified in any external description to avoid implying distributed/autonomous deployment
-

Phase 1 — Systems & Architecture

Components: rule author → oracle author (hard gate) → code-gen agent (structurally blocked without oracle reference) → validation agent + simulation agent (parallel, independent, neither sees the other's output) → linked result pair (binding_proof hash) → human approver.

No compliance-review component exists anywhere in this architecture. That absence is intentional and load-bearing.

Data contracts:

```
OracleArtifact:
  rule_id, rule_text, created_at
  test_cases: [{ applicant_id, applicant_data, expected_outcome,
author_rationale }]

GeneratedCode:
  rule_id, oracle_reference (hash/id only), source_code,
  generation_rationale, generated_at

CheckResult (validation | simulation, same shape):
  rule_id, check_type
  per_test_case: [{ applicant_id, oracle_expected, agent_observed, match,
rationale }]

LinkedResultPair:
  rule_id, code_hash, validation_result_ref, simulation_result_ref,
  binding_proof (hash of code_hash + both result refs),
  approved (bool|null), approved_at
```

Orchestration / failure policy:

Failure class	Applies to	Policy
Missing precondition	Oracle load	Hard halt, no retry
Transient technical failure	Code-gen, validation, simulation (each independent)	One identical-input retry, both attempts logged, halt only that path if both fail
Substantive disagreement	Validation/simulation vs. oracle	Not a failure — run to completion, surface prominently in final report

Validation and simulation crashes are isolated from each other — one crashing doesn't prevent the other from completing; the run halts afterward with whatever results exist.

Invocation: CLI (`run.py --rule-id <id>` or `run.py --all`). Isolated re-runs allowed but always regenerate code fresh — no caching, no reuse of prior generated output. Non-determinism is expected and acknowledged: a re-run is an independent roll, not a replay. Canonical demo artifacts come from one specific run, kept and reported as-is.

Phase 2 — Domain & API

Rule set (5 total, frozen):

Rule	Type
DTI > 43% → deny	Clean
Loan > \$50k → manual review	Clean
Credit score "above 750"	Boundary trap
"Recent late payment"	Undefined-term trap
Compound and/or condition	Precedence trap

Every rule's outcome is reported regardless of whether its trap triggers — non-triggers are reported as "resolved correctly," never omitted.

Rule entity includes `intended_semantics` — oracle-author's private note on what the rule *should* mean, authored before writing per-applicant test cases. This field has exactly one legitimate reader (the oracle-author, at authoring time) and no path to reach any agent — it is not part of the `OracleArtifact` schema at all.

API access boundaries:

Function	Sees	Explicitly excluded
<code>author_oracle</code>	rule_text, intended_semantics	—
<code>generate_code</code>	rule_text, oracle_reference (hash only)	intended_semantics, test_cases
<code>validate</code>	generated code, oracle (rule_text + test_cases)	intended_semantics
<code>simulate</code>	generated code, oracle (rule_text + test_cases)	intended_semantics

Governing principle: every agent's rationale must be reproducible from what that agent was actually given. No agent reasons from information another agent lacked, except code-gen's necessary blindness to oracle contents (by design).

Phase 3 — Scope & Production

Definition of done — all must be true simultaneously:

1. All 5 oracles frozen, authored before code-gen ran for that rule
2. `run.py --all` reaches a final state (mismatches don't count as non-completion)

3. At least one trap produced an unstaged mismatch (never edited post-hoc to force a trigger)
4. Final report shows all 5 outcomes, not just failures
5. Every `binding_proof` is independently verifiable
6. All 5 pairs actually reviewed and recorded by the human approver — not just run and glanced at

Deliverable: code + machine-generated report + README. README must report all 5 outcomes with the same no-skimming discipline as the machine report, and must restate the /v4 non-goals explicitly.

Build sequence:

1. Author oracles for the 2 clean rules only (thin first pass)
2. Build oracle-gate + code-gen for one clean rule — confirm hard-halt actually halts
3. Add `validate()` + `simulate()` for that same clean rule — confirm independence and `binding_proof` on known-boring ground truth
4. Author oracles for the 3 trap rules — **the actual hard test**
5. Run all 5 through the proven pipeline — first point you learn if traps trigger
6. Build report generator + README last, against real (possibly mixed) data

Risk register:

Risk	Response
"Clean" rule reveals hidden ambiguity while authoring its oracle	Reclassify as trap, report honestly — not a failure
LLM API flakiness	Retry-then-halt (structural, Phase 1)
Zero traps trigger	Valid evidence, reported as such; rule set frozen at 5, no post-hoc additions
Time runs short	Cut trap rule count before ever cutting oracle quality on a remaining rule
Oracle-author self-error	Before freezing each test case: re-derive expected outcome from <code>rule_text</code> alone, blind to your own intended <code>_semantics</code> — a different question than re-reading, not the same question asked twice

Demo exit criteria — success means a viewer can, unprompted, restate:

1. The thesis (structural separation, not "AI writes loan code")
2. Why oracle-before-codegen matters and what's lost without it

3. The actual outcome across all 5 rules, mismatches and correct-resolutions alike
4. What this does NOT prove — illustrated with a concrete worked example (e.g., a facially-neutral, discriminatory-in-effect rule that would pass this entire pipeline cleanly), not just a listed disclaimer

Post-demo artifacts:

- `LIMITATIONS.md` (external-facing): what this doesn't prove, the worked non-goal example, what real extension would actually require
 - `DESIGN_DECISIONS.md` (future-you-facing): tripwire list of specific shortcuts already considered and rejected —
 - Don't merge `validate()`/`simulate()` to save time — thesis violation, not a headcount decision
 - Don't let a re-run reuse prior generated code — this is how you'd accidentally cherry-pick a favorable result
 - Don't add a 6th rule if no trap triggers — frozen at 5 regardless of outcome
 - If a "clean" rule reveals hidden ambiguity, reclassify it — don't force it back to clean
-

What's still open

Nothing structural. This spec covers mission, architecture, data contracts, API boundaries, build order, risk responses, and exit criteria. What remains is content you own and haven't finalized: exact wording for the three trap rules, and the oracles themselves — both deliberately left for you to author, per the independence requirement this entire build exists to protect.